Formal Verification Report: Euler Vault Kit

- Repository: https://github.com/alexzoid-eth/euler-vault-cantina-fv
- Date: June 2024
- Author: @alexzoid
- Solidity Compiler: 0.8.23

Table of Contents

1. About Euler Vault Kit
2. Formal Verification Methodology
    - Types of Properties
    - Verification Process
    - Project Structure
    - Assumptions
3. Verification Properties
    - Valid State
    - Variable Transitions
    - State Transitions
    - High-Level
    - Unit Tests
4. Real Issues Found
5. Mutation Testing
    - What is Mutation Testing
    - Mutation Summary
6. Setup and Execution
    - Common Setup (Steps 1–4)
    - Remote Execution
    - Local Execution
    - Running Verification

About Euler Vault Kit

The Euler Vault Kit (EVK) is a modular lending vault system from the Euler protocol. Each EVault is an ERC-4626 compatible vault that supports lending, borrowing, and liquidation. The vault is composed of five modules that are deployed behind a proxy and interact through shared storage:

1. **Vault** — Core ERC-4626 deposit/mint/withdraw/redeem operations and share accounting

2. **Borrowing** — Borrow assets against collateral, repay debt, pull debt between accounts
3. **Governance** — Governor-controlled configuration: LTV parameters, fees, interest rate model, hooks, caps
4. **Liquidation** — Liquidate unhealthy positions with configurable discount and cool-off period
5. **RiskManager** — Enforce supply/borrow caps and account health via the Ethereum Vault Connector (EVC)

Key operations:

- `deposit` / `mint` / `withdraw` / `redeem` — ERC-4626 vault operations for supplying and withdrawing assets
- `borrow` / `repay` / `repayWithShares` / `pullDebt` — Borrowing operations with interest accumulation
- `liquidate` — Seize collateral from unhealthy accounts at a discount
- `convertFees` — Distribute accumulated interest fees to governor and protocol receivers
- `setLTV` / `setInterestRateModel` / `setCaps` / `setHookConfig` — Governor-only configuration

Formal Verification Methodology

Certora Formal Verification (FV) provides mathematical proofs of smart contract correctness by verifying code against a formal specification. Unlike testing and fuzzing which examine specific execution paths, Certora FV examines all possible states and execution paths.

The process involves crafting properties in CVL (Certora Verification Language) and submitting them alongside compiled Solidity smart contracts to a remote prover. The prover transforms the contract bytecode and rules into a mathematical model and determines the validity of rules.

Types of Properties

Properties are categorized following the [official Certora methodology]. Valid State, Variable Transitions and State Transitions properties are **parametric** — they are automatically verified against every external function in the contract, including functions added after the specification is completed. High-Level properties target specific function sequences.

Valid State — System-wide invariants that MUST always hold true. These properties define the fundamental constraints of the protocol, such as accounting consistency and structural integrity. Once proven, these invariants serve as trusted assumptions in other properties via `requireInvariant`, reducing verification complexity.

Variable Transitions — Properties that verify specific storage variables change only under expected conditions. The process captures a variable value before instruction execution, runs the instruction, then asserts the variable changed only as permitted or remained unchanged.

State Transitions — Properties that verify the correctness of transitions between valid states. Building upon the valid state invariants, these properties ensure the protocol's state machine operates correctly and that state changes are both authorized and sequentially valid.

High-Level — Complex multi-step properties verifying business logic integrity. Unlike parametric properties, these target specific function sequences to validate end-to-end protocol behavior.

Unit Tests — Properties that verify basic behavior of individual functions: revert conditions, direct effects on state, and non-effects on unrelated state. Unlike parametric properties, each unit test targets a specific function call.

Verification Process

1. **Setup phase**: Define ghost variables, storage hooks, and helper definitions to model contract state in CVL. Establish verification harnesses and configurations for five independent module targets (Vault, Borrowing, Governance, Liquidation, RiskManager). This phase addresses several prover limitations:
    - Storage modeling (`StorageHooks.spec`, `StorageHooksAsset.spec`): Ghost variables and storage hooks mirror all relevant vault storage slots — balances, borrows, accumulated fees, LTV configurations, interest accumulators, and hook settings — enabling CVL rules to reason about the full protocol state.
    - External dependency summaries (`Base.spec`): Complex external interactions (EVC, price oracles, interest rate models, protocol config, hook targets, balance trackers) are replaced with ghost-

backed CVL functions or `NONDET` summaries that preserve essential behavioral properties while eliminating implementation complexity.
    - ERC-20 model: Real ERC-20 contracts are handled via `DummyERC20A` and `DISPATCHER` summaries for standard token operations (`balanceOf`, `transfer`, `transferFrom`, `approve`), enabling tractable asset transfer reasoning.
    - Math utilities: `RPow` (exponentiation) is summarized via a CVL function `CVLPow`, and `mulDiv` is replaced with exact CVL arithmetic to avoid solver overflow.
    - Vault cache (`LoadVault.spec`): The `loadVault()` function is summarized to model vault state loading without triggering interest accumulation in every rule.
2. **Crafting Properties**: Write invariants and rules in CVL, starting with valid state invariants (which become trusted preconditions for other rules via `requireInvariant`), then variable/state transitions, and finally high-level and unit test properties. Common parametric rules (`Funcs.spec`) and state invariants (`State.spec`) are shared across all five modules.

Project Structure

```
certora/
├── confs/                                  # Prover configuration files
│   ├── Borrowing_Common_verified.conf      # Common rules on Borrowing module
│   ├── Borrowing_verified.conf             # Borrowing-specific rules
│   ├── Governance_Common_verified.conf     # Common rules on Governance module
│   ├── Governance_L1_violated.conf         # Violated rules (bug confirmation)
│   ├── Governance_verified.conf            # Governance-specific rules
│   ├── Liquidation_Common_verified.conf    # Common rules on Liquidation module
│   ├── Liquidation_verified.conf           # Liquidation-specific rules
│   ├── RiskManager_Common_verified.conf    # Common rules on RiskManager module
│   ├── RiskManager_verified.conf           # RiskManager-specific rules
│   ├── Vault_Common_verified.conf          # Common rules on Vault module
│   └── Vault_verified.conf                 # Vault-specific rules
│
├── harnesses/                              # Verification harnesses
│   ├── AbstractBaseHarness.sol             # Abstract base harness interface
│   ├── BaseHarness.sol                     # Base harness with shared helpers
│   └── modules/                            # Per-module harnesses
│       ├── BorrowingHarness.sol
│       ├── GovernanceHarness.sol
│       ├── LiquidationHarness.sol
│       ├── RiskManagerHarness.sol
│       └── VaultHarness.sol
│
├── helpers/                                # Helper contracts
│   ├── DummyERC20A.sol                     # Dummy ERC-20 for asset modeling
│   └── DummyERC20Impl.sol                  # ERC-20 implementation
│
├── mutations/                              # Mutation testing files
│   ├── AssetTransfers/                     # AssetTransfers.sol mutations
│   ├── BalanceUtils/                       # BalanceUtils.sol mutations
│   ├── Borrowing/                          # Borrowing.sol mutations
│   ├── BorrowUtils/                        # BorrowUtils.sol mutations
│   ├── Governance/                         # Governance.sol mutations
```

```
│   ├── Governance_L1_fix/                    # Fix for L1 bug
│   ├── Liquidation/                          # Liquidation.sol mutations
│   ├── RiskManager/                          # RiskManager.sol mutations
│   └── Vault/                                # Vault.sol mutations
│
├── scripts/                                  # Automation scripts
│   ├── add_mutation.sh
│   ├── certora_mutate_all.sh
│   ├── check_mutation_common.sh
│   └── check_mutation.sh
│
└── specs/                                    # CVL specifications
    ├── base/                                 # Shared setup: ghosts, hooks, summaries
    │   ├── Base.spec                         # External dependency summaries
    │   ├── LoadVault.spec                    # Vault cache loading summary
    │   ├── RPow.spec                         # Exponentiation CVL summary
    │   ├── StorageHooks.spec                 # Vault storage ghost variables and hooks
    │   ├── StorageHooksAsset.spec            # Asset (ERC-20) storage hooks
    │   └── methods/                          # Per-module method declarations
    │       ├── BaseMethods.spec
    │       ├── BorrowingMethods.spec
    │       ├── GovernanceMethods.spec
    │       ├── LiquidationMethods.spec
    │       ├── RiskManagerMethods.spec
    │       └── VaultMethods.spec
    ├── common/                               # Shared property definitions
    │   ├── Funcs.spec                        # Parametric rules (reentrancy, hooks, state
change)
    │   └── State.spec                        # Valid state invariants (ST-01 to ST-31)
    ├── Common.spec                           # Common cross-module rules (COM-01 to COM-
30)
    ├── Vault.spec                            # Vault module rules (VLT-01 to VLT-15)
    ├── Governance.spec                       # Governance module rules (GOV-01 to GOV-24)
    ├── Governance_L1_violated.spec           # Violated rules (L1, L1-EX)
    ├── Liquidation.spec                      # Liquidation module rules (LIQ-01 to LIQ-11)
    ├── RiskManager.spec                      # RiskManager module rules (RM-01 to RM-06)
    └── Borrowing.spec                        # Borrowing module rules (BRW-01 to BRW-08)
```

Assumptions

Formal verification requires assumptions about the code and its environment to address prover timeouts, tool limitations, and state consistency. However, incorrect assumptions can mask real bugs by excluding reachable states from analysis. To maintain transparency, all assumptions are documented below.

Safe Assumptions

These reflect real-world constraints that don't impact security guarantees.

Environment Constraints (`Base.spec`):

- Block timestamps fall within realistic bounds (`(0, 32499081600)` — up to year 3000)
- Invariant-preserved environments share the same timestamp

External Dependency Summaries

External contracts are replaced with ghost-backed CVL functions or `NONDET` summaries (`Base.spec`):

- **EVC**: `getCurrentOnBehalfOfAccount` returns a non-zero, non-self address via ghost; `requireVaultStatusCheck`, `requireAccountAndVaultStatusCheck` track calls via ghost booleans; `getLastAccountStatusCheckTimestamp` bounded by block timestamp; other EVC functions (`areChecksInProgress`, `disableController`, `isOperatorAuthenticated`, `controlCollateral`, `forgiveAccountStatusCheck`, `getControllers`, `getCollaterals`, `isCollateralEnabled`, `isAccountStatusCheckDeferred`, `isVaultStatusCheckDeferred`, `isControllerEnabled`) are `NONDET`
- **Price Oracle**: `getQuote` and `getQuotes` return values via ghost function with an upper bound (`< 2^230`) to prevent overflow in LTV calculations
- **Interest Rate Model**: `computeInterestRate` and `computeInterestRateView` use a ghost mapping `ghostComputedInterestRate[vault][cash][borrows]` to model deterministic rate computation
- **Protocol Config**: `protocolFeeConfig` returns ghost-backed `(address, uint16)` per vault; `isValidInterestFee` is `NONDET`
- **Hook Target**: `isHookTarget()` is `NONDET`; `invokeHookTarget` is tracked via ghost variable
- **Balance Tracker**: `balanceTrackerHook` is modeled via ghost variables tracking account, balance, and forfeit flag
- **DToken**: `emitTransfer` is `NONDET`
- **Flash Loan**: `onFlashLoan` is `NONDET`
- **ERC-20 Asset**: Standard functions (`name`, `symbol`, `decimals`, `totalSupply`, `balanceOf`, `allowance`, `approve`, `transfer`, `transferFrom`) use `DISPATCHER(true)` to delegate to `DummyERC20A`

Math Summaries

- `RPow.rpow(x, y, base)` replaced with CVL function `CVLPow` for tractable exponentiation
- `mulDiv(a, b, c)` replaced with exact CVL arithmetic `(a * b) / c` with `uint144` bound check
- `trySafeTransferFrom` / `safeTransferFrom` summarized as `DummyERC20A.transferFrom`
- `ProxyUtils.metadata()` returns ghost-backed oracle and unit-of-account addresses with the dummy ERC-20 as the asset

Proved Assumptions

These properties have been formally verified as valid state invariants (ST-01 to ST-31) and are used as trusted preconditions (via `requireInvariant`) in state transition and high-level rules. See Valid State for detailed descriptions and verification statuses.

The helper functions `requireValidStateCVL()`, `requireValidStateEnvCVL(e)`, `requireValidStateCollateralCVL(collateral)`, and `requireValidStateEnvCollateralCVL(e, collateral)` in `State.spec` bundle these invariants as preconditions for use in other rules.

Bounded Model

- Loop iterations capped at 2 across all configurations (`loop_iter: 2`, `optimistic_loop: true`)
- LTV list length bounded below `max_uint64` for the `initializedLTVInCollateralList` invariant

Verification Properties

Links to specific CVL spec files are provided for each property, with status indicators.

- ✅ Verified
- ❌ Violated

Valid State

Valid State properties define the fundamental invariants that must always hold true throughout the protocol's lifecycle. These are parametric — each property is automatically verified against every external function in the contract, including functions added in the future.

Common State Invariants

The 31 common invariants from `State.spec` are verified independently across all five modules (Vault, Borrowing, Governance, Liquidation, RiskManager).

Property	Name	Description	Status	Notes
ST-01	`vaultNotDeinitialized`	Vault MUST NOT be deinitialized	✅	
ST-02	`uninitializedSnapshotReset`	Uninitialized snapshot MUST be reset	✅	
ST-03	`snapshotStampAlwaysOne`	Snapshot stamp MUST be always equal to 1	✅	
ST-04	`timestampSetWhenPositiveAccumulatedFees`	Last interest accumulator timestamp set when positive accumulated fees	✅	
ST-05	`lastInterestAccumulatorNotInFuture`	Last interest accumulator timestamp MUST NOT be in the future	✅	
ST-06	`cashNotLessThanAssets`	Cash amount MUST NOT be less than the ERC20 assets stored in the current contract	✅	
ST-07	`supplyBorrowCapsLimits`	Max supply and borrow caps limitations	✅	
ST-08	`hooksLimits`	Hooks limitations	✅	
ST-09	`accumulatedFeesLimits`	Accumulated fees limitations	✅	
ST-10	`validateNotUseZeroAddress`	Shares cannot be transferred to the zero address	✅	
ST-11	`noSelfCollateralization`	Self-collateralization is not allowed	✅	
ST-12	`borrowLTVLowerOrEqualLiquidationLTV`	The borrow LTV MUST be lower than or equal to the liquidation LTV	✅	

Property	Name	Description	Status	Notes
ST-13	`initializedLTVWhenSet`	The LTV is always initialized when set	✅	
ST-14	`zeroTimestampInitializedSolvency`	LTV with zero timestamp should not be initialized	✅	
ST-15	`ltvTimestampValid`	LTV's timestamp is always less than or equal to the current timestamp	✅	
ST-16	`ltvTimestampFutureRamping`	LTV's timestamp MUST be in the future only when ramping set	✅	
ST-17	`initializedLTVInCollateralList`	Initialized LTV exists in collaterals list	✅	
ST-18	`zeroTimestampIndicatesLTVCleared`	Zero timestamp means the LTV is cleared or not set yet	✅	
ST-19	`configParamsScaledTo1e4`	Config parameters are scaled to `1e4`	✅	
ST-20	`uniqueLTVEntries`	All collateral entries in the vault storage LTV list MUST be unique	✅	
ST-21	`ltvFractionScaled`	The specified LTV is a fraction between 0 and 1 (scaled by 10,000)	✅	
ST-22	`ltvLiquidationDynamic`	Liquidation LTV is calculated dynamically only when ramping is in progress and always between the target liquidation LTV and the initial liquidation LTV	✅	
ST-23	`ltvRampingTimeWithinBounds`	When ramping is in progress, the time remaining is always less than or equal to the ramp duration	✅	
ST-24	`configFlagsLimits`	Config flags limitations	✅	
ST-25	`transferNotAllowedToZeroAddress`	Transfer assets to zero address not allowed	✅	
ST-26	`interestRateMaxLimit`	Interest rate has a maximum limit of 1,000,000 APY	✅	
ST-27	`userInterestAccumulatorLeqVault`	User interest accumulator always less or equal vault interest accumulator	✅	
ST-28	`userInterestAccumulatorSetWithNonZeroOwed`	User's interest accumulator set when non-zero owed	✅	
ST-29	`interestAccumulatorScaledBy1e27`	Interest accumulator is scaled by 1e27	✅	

Property	Name	Description	Status	Notes
ST-30	`interestRateZeroWithoutModel`	Interest rate zero when interest rate model contract is not set	✅	
ST-31	`differentOwnerAndSpenderAllowances`	Owner and spender in allowances should differ	✅	

Common Rules (Valid State)

Property	Name	Description	Status	Notes
COM-02	`snapshotUsedWhenInitialized`	Snapshot MUST NOT be used when it is not initialized	✅	
COM-14	`collateralLTVNotRemoved`	Collateral LTV MUST NOT be removed completely	✅	
COM-21	`creatorUnchanged`	Creator is unchanged	✅	
COM-22	`accumulatedFeesUnchangedWhenInterestFeesZero`	Accumulated fees unchanged when interest fees zero	✅	
COM-27	`userBorrowChangesInTotalBorrows`	User borrow changes must be reflected in total borrows	✅	

Vault

Property	Name	Description	Status	Notes
VLT-01	`accumulatedFeesLeqShares`	Accumulated fees MUST always be less than or equal to total shares	✅	
VLT-02	`userBalanceEqualTotalShares`	User balance plus accumulated fees MUST always be equal to the total shares (with only 1 user)	✅	
VLT-03	`sumOfUsersBalanceEqualTotalShares`	Sum of three users' balance MUST always be equal to the total shares (with only 3 users)	✅	
VLT-06	`snapshotDisabledWithBothCapsDisabled`	Snapshot is disabled if both caps are disabled (at low-level set to 0, but resolved to max_uint256)	✅	
VLT-08	`stateChangeFunctionsReentrancyProtected`	State change functions are protected against reentrancy	✅	
VLT-10	`specificViewFunctionsProtectedAgainstReentrancy`	Specific view functions are protected against reentrancy, while others are not	✅	
VLT-13	`hookExecutionAllowance`	Hook execution allowance	✅	
VLT-15	`hookExecutionRestriction`	Hook execution restriction	✅	

Governance

Property	Name	Description	Status	Notes
GOV-02	`onlyOneGovernorExists`	Only one governor can exist at one time	✅	
GOV-07	`viewFunctionsNotProtectedAgainstReentrancy`	View functions MUST NOT be protected against reentrancy	✅	
GOV-14	`governorOnlyNotAffectOtherMethods`	Non-governor methods MUST be accessible to any callers	✅	
GOV-17	`initialLiquidationLTVSolvency`	Initial liquidation LTV is always the previous liquidation LTV or greater than liquidation LTV when ramping	✅	
GOV-20	`stateChangeFunctionsReentrancyProtected`	State change functions are protected against reentrancy	✅	
GOV-21	`anyoneCanExecuteViewFunctions`	Anyone can execute view functions	✅	
GOV-22	`hookExecutionAllowance`	Hook execution allowance	✅	
GOV-24	`hookExecutionRestriction`	Hook execution restriction	✅	

RiskManager

Property	Name	Description	Status	Notes
RM-01	`totalSharesBorrowsLimits`	Total shares and total borrows limitations	✅	
RM-05	`specificViewFunctionsProtectedAgainstReentrancy`	Specific view functions are protected against reentrancy	✅	
RM-06	`hookExecutionRestriction`	Hook execution restriction	✅	

Liquidation

Property	Name	Description	Status	Notes
LIQ-04	`stateChangeFunctionsReentrancyProtected`	State change functions are protected against reentrancy	✅	
LIQ-07	`specificViewFunctionsProtectedAgainstReentrancy`	Specific view functions are protected against reentrancy	✅	
LIQ-09	`hookExecutionAllowance`	Hook execution allowance	✅	
LIQ-11	`hookExecutionRestriction`	Hook execution restriction	✅	

Borrowing

Property	Name	Description	Status	Notes
BRW-02	`stateChangeFunctionsReentrancyProtected`	State change functions are protected against reentrancy	✅	
BRW-05	`specificViewFunctionsProtectedAgainstReentrancy`	Specific view functions are protected against reentrancy	✅	
BRW-06	`hookExecutionAllowance`	Hook execution allowance	✅	
BRW-08	`hookExecutionRestriction`	Hook execution restriction	✅	

Variable Transitions

Variable Transition properties verify that specific storage variables change only under expected conditions (monotonicity, set-once semantics, or authorized-function restrictions). These are parametric — each property is automatically verified against every external function in the contract, including functions added in the future.

Common

Property	Name	Description	Status	Notes
COM-04	`feesClearedToReceivers`	Clearing accumulated fees MUST move the fees to one or two designated fee receiver addresses	✅	
COM-11	`accumulatedFeesNonDecreasing`	Accumulated fees MUST NOT decrease unless they are being reset to zero	✅	
COM-15	`interestAccumulatorAlwaysGrows`	Interest accumulator always grows	✅	
COM-16	`userInterestAccumulatorAlwaysGrows`	User interest accumulator always grows	✅	
COM-17	`userInterestAccumulatorSetOnBorrowChange`	User interest accumulator set always when user borrow changes	✅	
COM-18	`interestAccumulatorCannotOverflow`	Interest accumulator cannot overflow	✅	

State Transitions

State Transition properties verify the correctness of transitions between valid states. These are parametric — each property is automatically verified against every external function in the contract, including functions added in the future.

Common

Property	Name	Description	Status	Notes
COM-01	`accumulatedFeesIncreaseTotalShares`	Accumulated fees MUST result in an increase in the total shares of the vault	✅	
COM-03	`snapshotCashSetFromStorage`	Snapshot cash MUST set from storage cash or reset	✅	
COM-05	`notAbleReceiveNativeTokens`	Functions are not able to receive native tokens	✅	
COM-06	`interestFeesAccruedSetTimestamp`	Change interest accumulator or accumulated fees accrued MUST set last interest accumulator timestamp	✅	
COM-07	`feesAndInterestNotUpdateNoAccumulatorUpdate`	Interest accumulator is updated only when last interest accumulator time changed	✅	
COM-08	`cashChangesSkim`	The vault's cash changes without assets transfer only when surplus assets available	✅	
COM-12	`feesRetrievedForCurrentContract`	Fees are retrieved only for the contract itself from the protocol config contract	✅	
COM-19	`interestRateForCurrentContract`	Interest rate computed always for the current contract	✅	
COM-20	`transferAllowedOnlyWithCompatibleAsset`	Transfer assets to sub-account allowed only when asset is compatible with EVC	✅	
COM-23	`allowanceUnchangedSelfWithdraw`	Allowance unchanged when user redeem shares to their own account	✅	
COM-24	`allowanceChangedFromAnotherWithdraw`	Allowance decrease (unless equal to max_uint256) when user redeem from another account	✅	
COM-25	`onlyOwnerCanIncreaseAllowance`	Only the owner can increase allowance	✅	

Vault

Property	Name	Description	Status	Notes
VLT-07	`viewFunctionsDontUpdateState`	View functions don't update the state	✅	

Property	Name	Description	Status	Notes
VLT-09	`anyoneCanExecuteViewFunctions`	Anyone can execute view functions	✅	
VLT-11	`specificFunctionsModifyState`	Specific functions can modify state	✅	
VLT-12	`modifyStatePossibility`	Possibility of modifying state	✅	
VLT-14	`hookExecutionPossibility`	Hook execution possibility	✅	

Governance

Property	Name	Description	Status	Notes
GOV-06	`protocolFeeConfigCalled`	While distributing fees, external protocol config contract is used	✅	
GOV-08	`viewFunctionsDontUpdateState`	View functions don't update state	✅	
GOV-12	`feesDistributionClearAccumulatedFeesNotAffectTotalShares`	While distributing fees, total shares MUST NOT change and accumulated fees are cleared	✅	
GOV-13	`governorAndProtocolReceiveFees`	While distributing fees, shares are transferred to governor and protocol fee receiver addresses	✅	
GOV-16	`LTVUpdateImmediate`	LTV can be increased or decreased immediately	✅	
GOV-18	`specificFunctionsModifyState`	Specific functions can modify state	✅	
GOV-19	`modifyStatePossibility`	Possibility of modifying state	✅	
GOV-23	`hookExecutionPossibility`	Hook execution possibility	✅	

RiskManager

Property	Name	Description	Status	Notes
RM-02	`viewFunctionsDontUpdateState`	View functions don't update the state	✅	
RM-03	`specificFunctionsModifyState`	Specific functions can modify state	✅	
RM-04	`modifyStatePossibility`	Possibility of modifying state	✅	

Liquidation

Property	Name	Description	Status	Notes
LIQ-03	`viewFunctionsDontUpdateState`	View functions don't update the state	✅	

Property	Name	Description	Status	Notes
LIQ-05	`specificFunctionsModifyState`	Specific functions can modify state	✅	
LIQ-06	`modifyStatePossibility`	Possibility of modifying state	✅	
LIQ-10	`hookExecutionPossibility`	Hook execution possibility	✅	

Borrowing

Property	Name	Description	Status	Notes
BRW-01	`viewFunctionsDontUpdateState`	View functions don't update the state	✅	
BRW-03	`specificFunctionsModifyState`	Specific functions can modify state	✅	
BRW-04	`modifyStatePossibility`	Possibility of modifying state	✅	
BRW-07	`hookExecutionPossibility`	Hook execution possibility	✅	

High-Level

High-Level properties verify multi-step business logic, accounting consistency across operations, and protocol-wide security guarantees.

Common

Property	Name	Description	Status	Notes
COM-09	`transferOutDecreaseCash`	Transferring assets from the vault MUST decrease the available cash balance	✅	
COM-10	`cashChangesAffectTotalShares`	Changes in the cash balance MUST correspond to changes in the total shares	✅	
COM-13	`rampDurationOnlyWhenLoweringLiquidationLTV`	Ramp duration can be used only when lowering liquidation LTV	✅	
COM-26	`balanceForwarderExecutedOnShareMoves`	Balance forwarder must be executed on any share movements when set	✅	
COM-28	`changeUserBorrowChangeVaultAssetBalance`	Increase or decrease user borrow transfers vault's assets out or in	✅	
COM-29	`sharesDecreaseOrBorrowsIncreaseExecutesAccountHealthCheck`	Any shares or borrows movements required account health check	✅	
COM-30	`sharesIncreaseOrBorrowsIncreaseExecutesVaultHealthCheck`	Increase shares or increase borrows MUST execute VAULT health check	✅	

Vault

Property	Name	Description	Status	Notes
VLT-04	`cashChangesWithTransfer`	The vault's cash changes MUST be accompanied by assets transfer (when no surplus assets available)	✅	
VLT-05	`cashChangesAffectUserShares`	Changes in the cash balance MUST correspond to changes in user's shares	✅	

Governance

Property	Name	Description	Status	Notes
GOV-01	`governorOnlyMethods`	Only the governor can invoke methods that modify the configuration of the vault	✅	
GOV-10	`protocolGetsAllFeesIfGovernorReceiverNotSet`	If the governor receiver was not set, the protocol gets all fees	✅	
GOV-11	`maxProtocolFeeShareEnforced`	Protocol's fee share cannot be more than the max protocol fee share (50%)	✅	

Liquidation

Property	Name	Description	Status	Notes
LIQ-01	`liquidationCoolDownPeriodEnforced`	Liquidation operations are prohibited until the cool-down period has passed	✅	
LIQ-02	`checkLiquidationHealthy`	Check liquidation healthy	✅	
LIQ-08	`liquidationPossibility`	Possibility of liquidation	✅	

Unit Tests

Unit Test properties verify basic behavior of individual functions: revert conditions, direct effects on state, and non-effects on unrelated state.

Governance

Property	Name	Description	Status	Notes
GOV-03	`ownershipCanBeTransferred`	Governor's ownership can be transferred	✅	

Property	Name	Description	Status	Notes
GOV-04	`ownershipCanBeRevoked`	The ownership could be revoked by setting the governor to zero address	✅	
GOV-05	`feeReceiverCanBeTransferred`	The fee receiver address can be changed	✅	
GOV-09	`governorFeeReceiverCanBeDisabled`	The governor fee receiver can be disabled	✅	
GOV-15	`setLTVPossibility`	The LTV can be set for a collateral asset, including borrow LTV, liquidation LTV, and ramp duration	✅	

Real Issues Found

One real bug was identified through formal verification: the `protocolFeeShare()` getter returns an unbounded value from the protocol config, while the actual fee distribution logic in `convertFees()` caps it at `MAX_PROTOCOL_FEE_SHARE` (50%). This creates an inconsistency where the view function reports a higher protocol fee share than what is actually applied.

L1: `protocolFeeShare()` displays incorrect value [Unit Test]

The `protocolFeeShare()` view function returns the raw protocol share from the protocol config contract without enforcing the `MAX_PROTOCOL_FEE_SHARE` (50%) cap. However, `convertFees()` correctly caps the protocol share before distributing. This means the getter returns a misleading value when the configured protocol share exceeds 50%.

❌ Violated:

```
// L1 | `protocolFeeShare()` displays incorrect value as it doesn't consider 50% limitation
rule protocolFeeShareLimited(env e) {
    assert(to_mathint(protocolFeeShare(e)) <= MAX_PROTOCOL_FEE_SHARE());
}
```

L1-EX: Protocol fee receiver gets wrong share amount [High Level]

Extends L1 to prove the accounting consequence: when `protocolFeeShare()` reports a value higher than 50%, the actual shares distributed to the protocol receiver differ from what the getter predicts.

❌ Violated:

```
// L1-EX | Ensure that protocol fee receiver gets an amount of shares equal to what
//          `protocolFeeShare()` getter returns
rule protocolFeeShareLimitedEx(env e, method f, calldataarg args) {
    address governor = feeReceiver();
    address protocol = ghostProtocolFeeReceiver[currentContract];
    require(governor != 0);
    require(governor != protocol);

    mathint getterFeeShare = to_mathint(protocolFeeShare(e));
    mathint governorExpectShares = (ghostAccumulatedFees * (CONFIG_SCALE() -
getterFeeShare)) / CONFIG_SCALE();
    mathint protocolExpectShares = ghostAccumulatedFees - governorExpectShares;

    mathint protocolBalanceBefore = ghostUsersDataBalance[protocol];
    f(e, args);
    mathint protocolBalanceAfter = ghostUsersDataBalance[protocol];

    assert(protocolBalanceAfter != protocolBalanceBefore
        => protocolBalanceAfter - protocolBalanceBefore == protocolExpectShares
    );
}
```

✅ Passed (after the fix):

Cap the return value of `protocolFeeShare()` to match the behavior of `convertFees()`:

```
 function protocolFeeShare() public view virtual reentrantOK returns (uint256) {
     (, uint256 protocolShare) = protocolConfig.protocolFeeConfig(address(this));
-    return protocolShare;
+    return protocolShare > MAX_PROTOCOL_FEE_SHARE ? MAX_PROTOCOL_FEE_SHARE : protocolShare;
 }
```

The fix is available in `mutations/Governance_L1_fix/1.sol`. After applying the fix, both L1 and L1-EX pass verification.

Mutation Testing

What is Mutation Testing

Mutation testing validates the strength of a formal specification by injecting small syntactic changes (mutations) into the source code and verifying that the spec catches them. Each mutation represents a potential bug — a removed assignment, a flipped operator, an off-by-one error.

The certoraMutate tool automates this process: it applies each mutant to the source, runs the prover against the specified rule, and reports whether the invariant caught the introduced defect.

Mutation Summary

Each configuration file defines mutation targets — source files paired with directories of mutant variants. The following table summarizes all mutation targets across modules:

Configuration	Source File	Mutations Directory
`Vault_verified.conf`	`src/EVault/modules/Vault.sol`	`mutations/Vault`
`Vault_verified.conf`	`src/EVault/shared/BalanceUtils.sol`	`mutations/BalanceUtils`
`Vault_verified.conf`	`src/EVault/shared/AssetTransfers.sol`	`mutations/AssetTransfers`
`Vault_Common_verified.conf`	`src/EVault/modules/Vault.sol`	`mutations/Vault`
`Vault_Common_verified.conf`	`src/EVault/shared/BalanceUtils.sol`	`mutations/BalanceUtils`
`Vault_Common_verified.conf`	`src/EVault/shared/AssetTransfers.sol`	`mutations/AssetTransfers`
`Governance_verified.conf`	`src/EVault/modules/Governance.sol`	`mutations/Governance`
`Governance_verified.conf`	`src/EVault/shared/BalanceUtils.sol`	`mutations/BalanceUtils`
`Governance_verified.conf`	`src/EVault/shared/BorrowUtils.sol`	`mutations/BorrowUtils`
`Governance_Common_verified.conf`	`src/EVault/modules/Governance.sol`	`mutations/Governance`
`Governance_Common_verified.conf`	`src/EVault/shared/BalanceUtils.sol`	`mutations/BalanceUtils`
`Governance_Common_verified.conf`	`src/EVault/shared/BorrowUtils.sol`	`mutations/BorrowUtils`
`Governance_L1_violated.conf`	`src/EVault/modules/Governance.sol`	`mutations/Governance_L1_fix`
`Borrowing_verified.conf`	`src/EVault/modules/Borrowing.sol`	`mutations/Borrowing`
`Borrowing_verified.conf`	`src/EVault/shared/BalanceUtils.sol`	`mutations/BalanceUtils`
`Borrowing_verified.conf`	`src/EVault/shared/BorrowUtils.sol`	`mutations/BorrowUtils`
`Borrowing_verified.conf`	`src/EVault/shared/AssetTransfers.sol`	`mutations/AssetTransfers`
`Borrowing_Common_verified.conf`	`src/EVault/modules/Borrowing.sol`	`mutations/Borrowing`
`Borrowing_Common_verified.conf`	`src/EVault/shared/BalanceUtils.sol`	`mutations/BalanceUtils`
`Borrowing_Common_verified.conf`	`src/EVault/shared/BorrowUtils.sol`	`mutations/BorrowUtils`
`Borrowing_Common_verified.conf`	`src/EVault/shared/AssetTransfers.sol`	`mutations/AssetTransfers`
`Liquidation_verified.conf`	`src/EVault/modules/Liquidation.sol`	`mutations/Liquidation`
`Liquidation_verified.conf`	`src/EVault/shared/BalanceUtils.sol`	`mutations/BalanceUtils`
`Liquidation_verified.conf`	`src/EVault/shared/BorrowUtils.sol`	`mutations/BorrowUtils`

Configuration	Source File	Mutations Directory
`Liquidation_Common_verified.conf`	`src/EVault/modules/Liquidation.sol`	`mutations/Liquidation`
`Liquidation_Common_verified.conf`	`src/EVault/shared/BalanceUtils.sol`	`mutations/BalanceUtils`
`Liquidation_Common_verified.conf`	`src/EVault/shared/BorrowUtils.sol`	`mutations/BorrowUtils`
`RiskManager_verified.conf`	`src/EVault/modules/RiskManager.sol`	`mutations/RiskManager`
`RiskManager_verified.conf`	`src/EVault/shared/BorrowUtils.sol`	`mutations/BorrowUtils`
`RiskManager_Common_verified.conf`	`src/EVault/modules/RiskManager.sol`	`mutations/RiskManager`
`RiskManager_Common_verified.conf`	`src/EVault/shared/BorrowUtils.sol`	`mutations/BorrowUtils`

Notable mutation-backed properties from `README_.md`:

Property	Mutation
COM-10	Borrowing/Borrowing_0.sol
ST-06	Vault/Vault_0.sol
ST-25	AssetTransfers/AssetTransfers_0.sol
GOV-06	Governance/1.sol
GOV-07	Governance/5.sol
GOV-18	Governance/2.sol
GOV-20	Governance/3.sol
GOV-21	Governance/6.sol

To run mutation testing for a specific configuration:

```
certoraMutate certora/confs/Vault_verified.conf
certoraMutate certora/confs/Governance_verified.conf
```

Or run all mutations using the provided script:

```
./certora/scripts/certora_mutate_all.sh
```

Setup and Execution

The Certora Prover can be run either remotely (using Certora's cloud infrastructure) or locally (building from source). Both modes share the same initial setup steps.

Common Setup (Steps 1–4)

The instructions below are for Ubuntu 24.04. For step-by-step installation details refer to this setup tutorial.

1. Install Java (tested with JDK 21)

```
sudo apt update
sudo apt install default-jre
java -version
```

2. Install pipx — installs Python CLI tools in isolated environments, avoiding dependency conflicts

```
sudo apt install pipx
pipx ensurepath
```

3. Install Certora CLI

```
pipx install certora-cli
```

4. Install solc-select and the Solidity compiler version required by the project

```
pipx install solc-select
solc-select install 0.8.23
solc-select use 0.8.23
```

Remote Execution

5. Set up Certora key. You can get a free key through the Certora Discord or on their website. Once you have it, export it:

```
echo "export CERTORAKEY=<your_certora_api_key>" >> ~/.bashrc
```

> **Note:** If a local prover is installed (see below), it takes priority. To force remote execution, add the `--server production` flag:
>
> ```
> certoraRun certora/confs/Vault_verified.conf --server production
> ```

Local Execution

Follow the full build instructions in the [CertoraProver repository](). Once the local prover is installed it takes priority over the remote cloud by default.

Running Verification

Build the project first:

```
forge build
```

Module-Specific Rules

Each module has its own configuration file for module-specific rules:

```
certoraRun certora/confs/Vault_verified.conf
certoraRun certora/confs/Governance_verified.conf
certoraRun certora/confs/Borrowing_verified.conf
certoraRun certora/confs/Liquidation_verified.conf
certoraRun certora/confs/RiskManager_verified.conf
```

Common Rules (cross-module)

Common rules from `Common.spec` are verified against each module independently:

```
certoraRun certora/confs/Vault_Common_verified.conf
certoraRun certora/confs/Governance_Common_verified.conf
certoraRun certora/confs/Borrowing_Common_verified.conf
certoraRun certora/confs/Liquidation_Common_verified.conf
certoraRun certora/confs/RiskManager_Common_verified.conf
```

Violated Rules (Bug Confirmation)

To verify the L1 bug and confirm the fix:

```
certoraRun certora/confs/Governance_L1_violated.conf
```

Mutation Testing

Run mutation testing for a specific configuration:

```
certoraMutate certora/confs/Vault_verified.conf
certoraMutate certora/confs/Governance_verified.conf
```

Or run all mutations:

```
./certora/scripts/certora_mutate_all.sh
```